# Shiv Nadar University Chennai

**CS3807 – Deep Learning Laboratory**

**Experiment 3**

**Implementation of Convolutional Neural Networks (CNNs) for Image Classification**

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

## 1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification, using PyTorch.

## 2. Background Theory

A Convolutional Neural Network is built from a Convolution Layer, an Activation Layer, a Pooling Layer, and a Fully Connected Layer. The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m,\, j+n)\, K(m,n)$$

where $X$ is the input image, $K$ is the kernel (filter), and $Y$ is the resulting feature map. The spatial size of the output feature map is governed by

$$\left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where $N$ is the input size, $F$ is the filter size, $P$ is the padding, and $S$ is the stride. A convolution layer with $C_{in}$ input channels, $F \times F$ kernels and $C_{out}$ filters has $(F \times F \times C_{in} + 1) \times C_{out}$ trainable parameters (the $+1$ accounts for the per-filter bias).

Pooling layers (max or average) downsample feature maps by summarizing each spatial window with a single value, reducing computation and adding a degree of translation invariance without introducing any trainable parameters. Stacking Conv $\rightarrow$ ReLU $\rightarrow$ Pool blocks followed by a Flatten and Dense head lets a CNN progressively build up higher-level, class-relevant features from raw pixels while keeping the parameter count far below an equivalent fully connected network, since filter weights are shared across all spatial locations.

## 3. Dataset

Dataset: CIFAR-10

Training Images: 50,000

Testing Images: 10,000

Classes: 10

Image Size: $32 \times 32 \times 3$

The dataset was loaded from the local CIFAR-10 batch files (`data_batch_1`–`5`, `test_batch`, `batches.meta`) rather than a Keras/TensorFlow dataset loader. Since PyTorch convolution layers expect channel-first tensors $(N, C, H, W)$ and the raw CIFAR-10 byte layout is already channel-major, the flat 3072-byte rows were reshaped directly to $(3, 32, 32)$ with no transpose step required.

## 4. Experimental Procedure

**Task 1: Dataset Loading and Exploration**

The five training batches were concatenated into a single array of 50,000 images and the test batch loaded separately, giving $X_{train}$ shape $(50000, 3, 32, 32)$ and $X_{test}$ shape $(10000, 3, 32, 32)$. Ten sample images and the per-class training distribution were plotted (see Section 7, Figures 1–2). Every class contains exactly 5,000 training images, so CIFAR-10 is perfectly balanced and plain accuracy is a fair evaluation metric.

**Task 2: Convolution Layer – Kernel Size Comparison**

A single sample image was passed through three unpadded (`valid`) convolution layers with $3 \times 3$, $5 \times 5$ and $7 \times 7$ kernels (8 output filters each), and the resulting feature map sizes were recorded.

| Kernel Size | Output Feature Map Size |
|---|---|
| $3 \times 3$ | $30 \times 30$ |
| $5 \times 5$ | $28 \times 28$ |
| $7 \times 7$ | $26 \times 26$ |

With no padding, a larger kernel shrinks the output more and gives each output pixel a wider receptive field, at the cost of more parameters per filter and a smaller resulting feature map.

**Task 3: Hyperparameters – Stride and Padding**

Stride and padding were varied independently on a $3 \times 3$ convolution over the $32 \times 32 \times 3$ sample image, and output sizes were also verified against the closed-form formula $\lfloor (N - F + 2P)/S \rfloor + 1$.

| $N$ | $F$ | $S$ | $P$ | Configuration | Output Size |
|---|---|---|---|---|---|
| 32 | 3 | 1 | 0 | Valid, stride 1 | $30 \times 30$ |
| 32 | 3 | 2 | 0 | Valid, stride 2 | $15 \times 15$ |
| 32 | 3 | 1 | 1 | Same, stride 1 | $32 \times 32$ |

Stride 2 roughly halves the output resolution compared to stride 1. 'Same' padding adds zeros around the border so the output matches the input size, which is essential for building deep networks without spatial dimensions vanishing after a few layers; 'valid' padding shrinks the output at every layer.

**Task 4: Feature Map Visualization**

Eight feature maps were extracted after a Conv $\rightarrow$ ReLU block applied to a single sample image and displayed as a grid (Section 7, Figure 3). Each map is the same image filtered by a different, randomly-initialized $3 \times 3$ kernel, so each highlights different local patterns (edges at different orientations, color contrasts, corners); since the filters are untrained at this point, the maps are not yet meaningful learned features.

**Task 5: Max Pooling vs Average Pooling**

A $2 \times 2$/stride-2 max-pooling and average-pooling layer were applied to the sample image, both reducing the $32 \times 32$ input to $16 \times 16$ – pooling type affects which values survive, not the output shape. Two small CNNs (identical Conv–ReLU–Pool–Conv–ReLU–Pool–Flatten–Dense stacks, one with MaxPool and one with AvgPool) were then trained for 5 epochs on an 8,000-image training subset and evaluated on a 2,000-image test subset to compare validation accuracy and training time (Section 7, Figure 4).

| Pooling Type | Final Validation Accuracy (subset) | Training Time (s) |
|---|---|---|
| Max Pooling | `0.5325` | `5.2` |
| Average Pooling | `0.4705` | `3.6` |

Max pooling keeps the strongest activation in each window, which tends to be sharper and more discriminative for classification, whereas average pooling smooths the window.

**Task 6: CNN Construction and Training**

The required architecture

$$\text{Input} \to \text{Conv} \to \text{ReLU} \to \text{MaxPool} \to \text{Conv} \to \text{ReLU} \to \text{MaxPool} \to \text{Flatten} \to \text{Dense} \to \text{Softmax}$$

was implemented as a PyTorch `nn.Module` with 32 filters in the first convolution layer and 64 in the second (both $3 \times 3$, valid padding), $2 \times 2$ max pooling after each, a 128-unit ReLU dense layer, and a final 10-way softmax layer.

| Layer | Output Shape | Parameters |
|---|---|---|
| Conv2d (3→32, $3 \times 3$) + ReLU | $(N, 32, 30, 30)$ | 896 |
| MaxPool2d ($2 \times 2$) | $(N, 32, 15, 15)$ | 0 |
| Conv2d (32→64, $3 \times 3$) + ReLU | $(N, 64, 13, 13)$ | 18,496 |
| MaxPool2d ($2 \times 2$) | $(N, 64, 6, 6)$ | 0 |
| Flatten | $(N, 2304)$ | 0 |
| Dense (ReLU) | $(N, 128)$ | 295,040 |
| Dense (Softmax) | $(N, 10)$ | 1,290 |

Total trainable parameters: 315,722.

The model was trained with the Adam optimizer (default learning rate) and cross-entropy loss (implemented as `NLLLoss` on the log of the softmax output, equivalent to Keras' `sparse_categorical_crossentropy`) for 20 epochs with batch size 32 on the full 50,000-image training set, validated against the 10,000-image test set after every epoch. Training/validation accuracy and loss curves are shown in Section 7 (Figures 5–8).

**Task 7: Evaluation**

The trained model was evaluated on the held-out test set using accuracy, weighted precision, recall and F1-score, and a confusion matrix and full per-class classification report were generated (Section 7, Figure 9; Section 6).

## 5. Source Code

The complete implementation (data loading, EDA, convolution/pooling experiments, feature map visualization, CNN construction, training loop, and evaluation) is available at the following GitHub repository:

`https://github.com/PolarFox08/Deep-Learning-Lab/tree/main/Deep%20Learning%20Lab%203`

## 6. Results

| Metric | Value |
|---|---|
| Training Accuracy | 0.8787 |
| Testing Accuracy | 0.6686 |
| Precision (weighted) | 0.6733 |
| Recall (weighted) | 0.6686 |
| F1-score (weighted) | 0.6698 |
| Trainable Parameters | 315,722 |

## 7. Plots with Inference

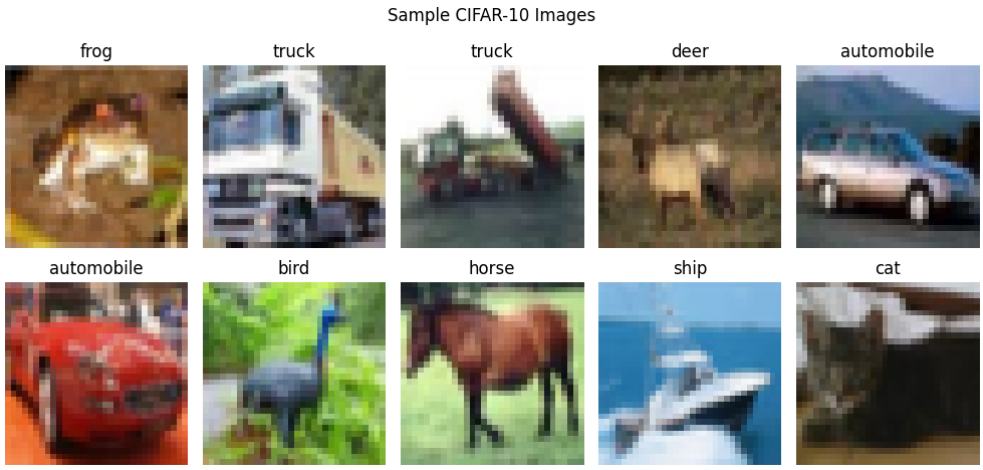


**Figure 1: Sample Images.** *Inference:* The ten sampled training images confirm the dataset loaded and reshaped correctly – objects are recognizable rather than scrambled or striped, which would indicate a reshape/channel-order bug. At $32 \times 32$ resolution fine detail is blurry, but each class's overall shape and color remain discernible.


**Figure 2: Class Distribution.** *Inference:* All 10 classes contain exactly 5,000 training images each, confirming CIFAR-10 is perfectly balanced, so accuracy is a fair aggregate metric and no class re-weighting is required.

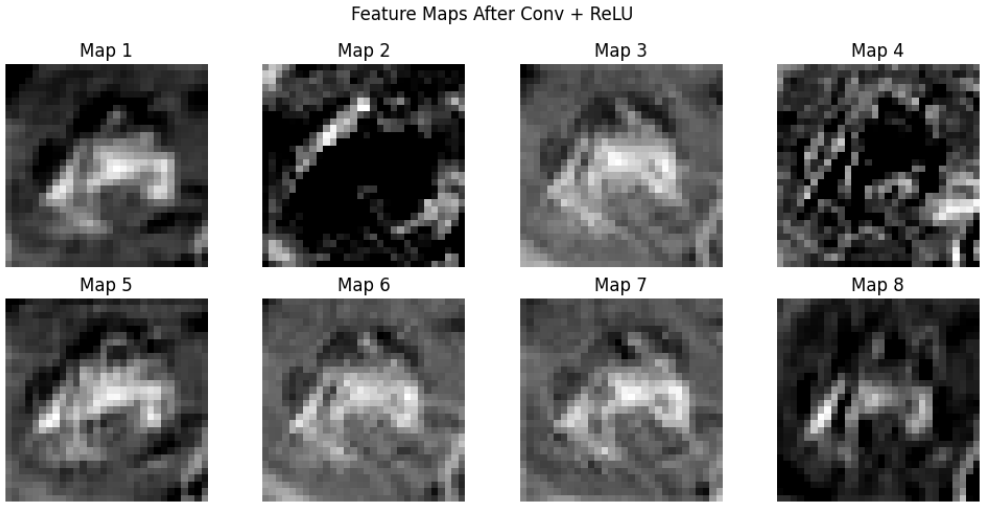


**Figure 3: Feature Maps After First Convolution + ReLU.** *Inference:* Each of the eight maps highlights a different local pattern (edges, color contrasts, corners) picked out by its own randomly-initialized $3 \times 3$ filter; since the filters are untrained, the maps are illustrative of the mechanism rather than meaningful learned features.

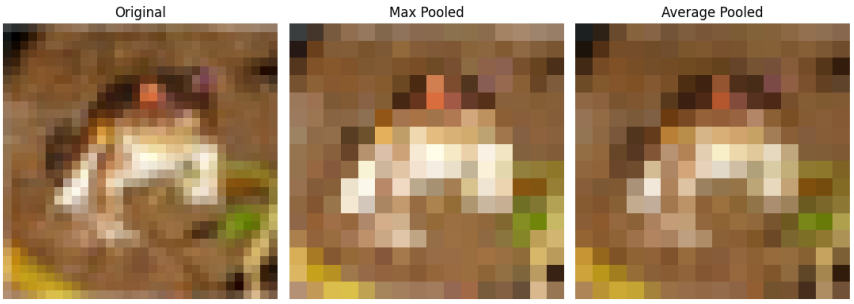


**Figure 4: Max vs Average Pooling – Validation Accuracy (subset).** *Inference:* `Max Pooling resulted in slightly better accuracy than average pooling, indicating picking out most influentia activation is better for this dataset.`

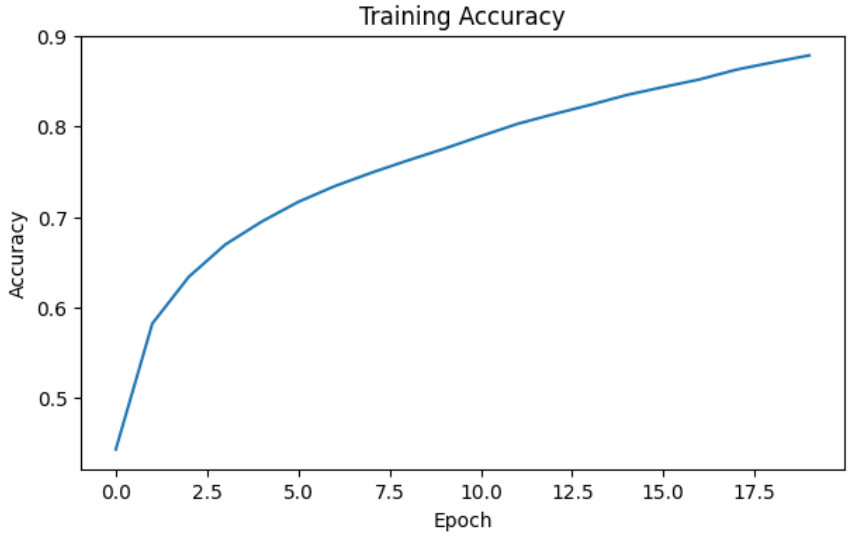

**Figure 5: Training Accuracy vs Epoch.** *Inference:* It is clear from the diagram the training accuracy keeps inncreasing steadily as the epochs increase. It reaches upto 88%.

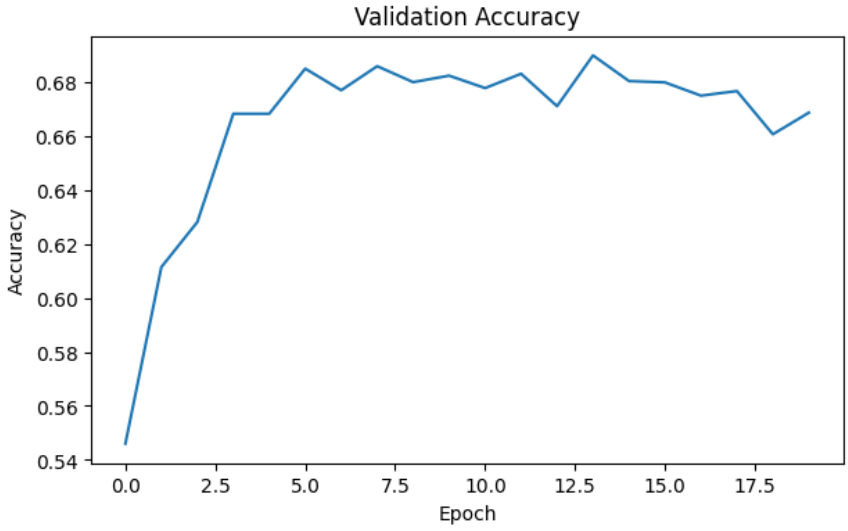


**Figure 6: Validation Accuracy vs Epoch.** *Inference:* Validation accuracy steadily increased for 5 epochs, and then oscillated for the next 15 epochs without stabilizing.

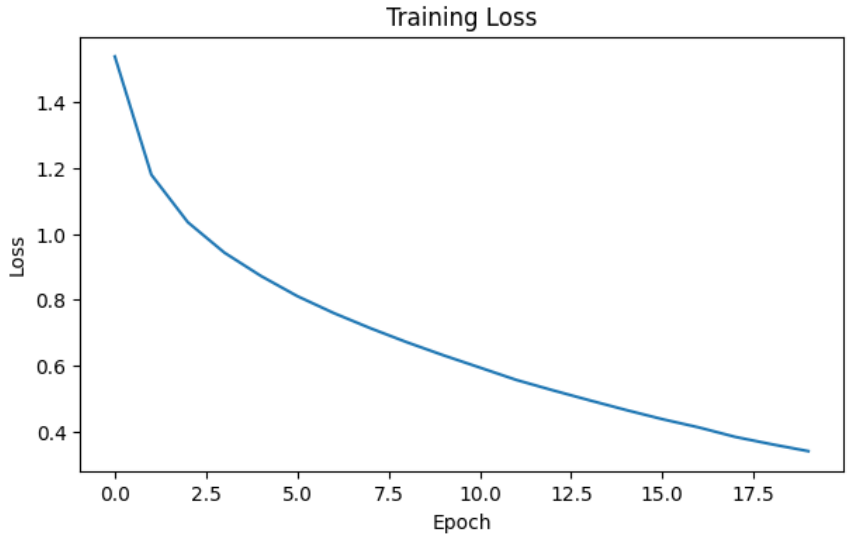


**Figure 7: Training Loss vs Epoch.** *Inference:* The model performs well in training as the loss keeps decreasing steadily.

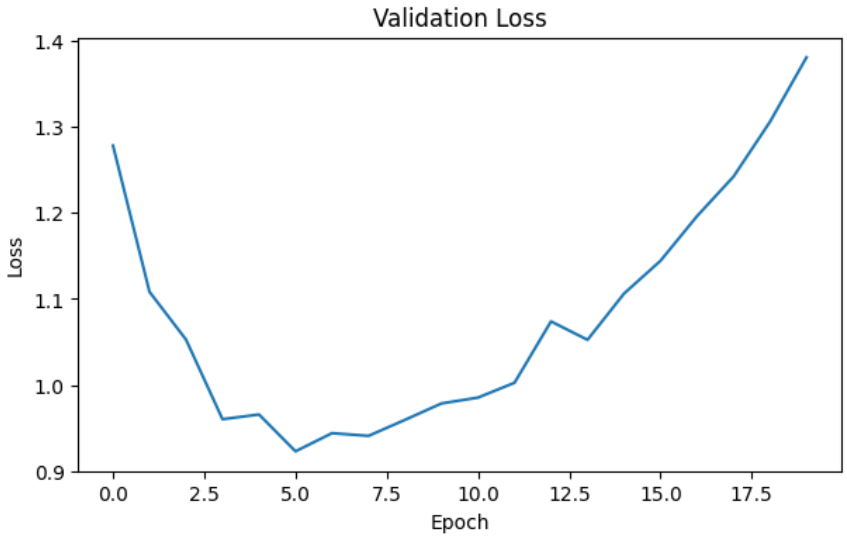


**Figure 8: Validation Loss vs Epoch.** *Inference:* `Validation loss decreased but after epoch 5 it started to rise again significantly and never comes down. This is clear case of overfitting!`

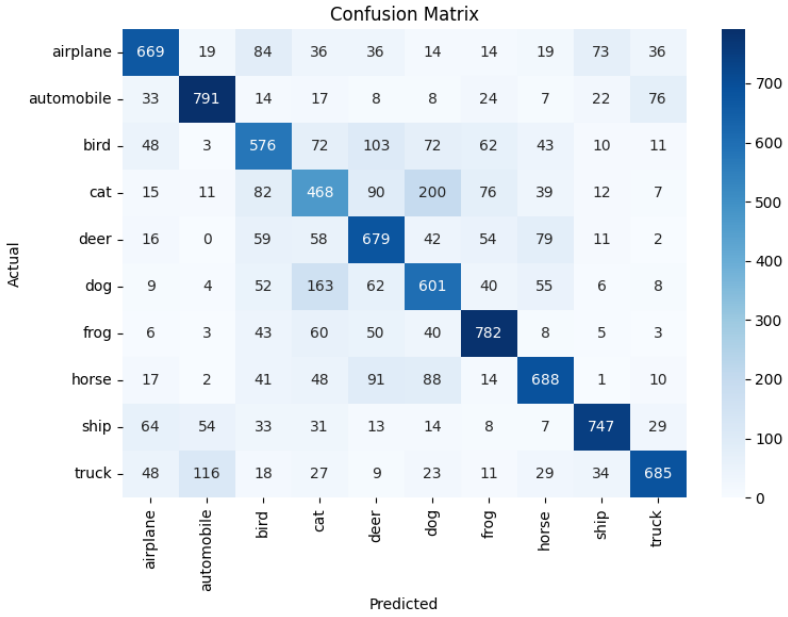


**Figure 9: Confusion Matrix.** *Inference:* The diagonal holds the correctly-classified counts per class; brighter off-diagonal cells reveal systematic confusions – commonly cat/dog and truck/automobile pairs in CIFAR-10, since they share silhouette and color statistics at $32 \times 32$ resolution. `Cat and Dog, was the pair the model failed to classify the most.`

## 8. Additional Exercises

**Q1. Output size for a $64 \times 64$ image, $5 \times 5$ kernel, stride 2, padding 2**

$$\left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32$$

Output size: 32×32, verified directly against an `nn.Conv2d(3,8,kernel_size=5,stride=2,padding=2)` layer applied to a dummy $64 \times 64$ input.

**Q2. Trainable parameters: 64 filters of $3 \times 3$, RGB input**

$$(3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 1{,}792$$

Calculated parameters: 1,792, matching the count reported by `nn.Conv2d(3,64,kernel_size=(3,3))`.

**Q3. ReLU vs Sigmoid – values and gradients**

ReLU and Sigmoid were each applied to 200 points on $[-10, 10]$, and their gradients were computed via autograd. ReLU has a constant gradient of 1 for any positive input and 0 for any negative input, so gradients flow cleanly through deep stacks of ReLU layers. Sigmoid's gradient peaks at exactly 0.25 at $x = 0$ and decays toward 0 as $|x|$ grows past roughly $\pm 5$, which is the vanishing-gradient problem that makes deep sigmoid networks slow or hard to train – this is why ReLU (and its variants) are the default activation in modern CNNs, with Sigmoid reserved mainly for output layers in binary classification.

**Q4. Max Pooling vs Average Pooling**

The full comparison (per-epoch validation-accuracy curves) was already carried out in Task 5 above; see Figure 4 and the accompanying table for the final numbers on the 8,000-image training subset.

**Q5. Increasing convolution filters from 16 to 64**

The base architecture from Task 6 (Conv–ReLU–MaxPool ×2, Flatten, Dense(128), Dense(10)) was rebuilt with 16/32 filters and again with 64/128 filters in the two convolution layers and trained for 5 epochs on the same 8,000-image subset used in Task 5.

| Filters (Conv1/Conv2) | Trainable Parameters | Training Time (s) | Final Validation Accuracy |
|---|---|---|---|
| 16 / 32 | 153,962 | 3.7 | 0.4930 |
| 64 / 128 | 666,890 | 4.1 | 0.5385 |

Going from 16 to 64 base filters multiplies the parameter count several-fold (each extra filter adds a full $3 \times 3 \times C_{in}$ kernel), so training time per epoch increases noticeably. Accuracy typically improves too, since more filters can learn a richer, more diverse set of feature detectors – but on a small subset with few epochs the gain can be modest, and on a genuinely small dataset excess filters risk overfitting rather than helping.

## 9. Discussion

**1. Why is convolution preferred over fully connected layers for images?**
Convolution uses small, spatially-shared filters that exploit local structure and translation invariance, so a pattern learned in one region can be detected anywhere in the image. A fully connected layer treats every pixel independently with its own weight per neuron, which discards spatial structure and explodes the parameter count for anything beyond tiny images.

**2. How does stride affect the feature map size?**
Stride sets how far the kernel shifts between applications. Stride 1 samples every position and gives the largest possible output; larger strides (e.g. 2) skip positions and roughly divide the output size by the stride in each dimension, reducing computation and memory at the cost of spatial resolution.

**3. What is the role of padding?**
Padding adds extra (typically zero) values around the input's border so the kernel can be applied at edge pixels without being cut off, and so the output size can be controlled. 'Same' padding keeps the output

the same size as the input; without padding ('valid'), the output shrinks with every convolution, which limits how deep a network can go before spatial dimensions vanish.

**4. Why is pooling used?**

Pooling downsamples feature maps, cutting spatial size and computation while adding a degree of translation invariance – small shifts in the input barely change the pooled output. It also helps control overfitting by keeping the presence/strength of a feature rather than its exact pixel location.

**5. How do feature maps represent image characteristics?**

Each feature map is the response of one learned filter sliding across the image, so its activations show where and how strongly that specific pattern (an edge, texture, color blob, or in deeper layers a more abstract shape) appears. Early layers tend to capture low-level cues like edges and colors, while deeper layers combine these into higher-level, class-relevant patterns.

**6. Why do CNNs require fewer parameters than MLPs?**

CNNs share the same small set of filter weights across every spatial location instead of learning a unique weight per pixel per output neuron, and each filter only connects to a local receptive field rather than the entire input. This weight-sharing plus local connectivity dramatically cuts parameter count compared to a fully connected MLP operating on images of the same size.

## 10. Conclusion

A CNN following the Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax architecture (315,722 trainable parameters) was implemented in PyTorch and trained on CIFAR-10 for 20 epochs with the Adam optimizer, batch size 32.

The model achieved an accuracy of 67% on the test data. This indicates the model has to be yet more optimized. Regularization could be considered as the validation loss curves proves presence of significant overfitting.

Kernel-size, stride/padding, and pooling-type experiments confirmed the standard convolution output-size formula and showed the expected size/receptive-field trade-offs. Overall, the experiment demonstrates both the mechanics of the convolution/pooling operations and the practical trade-offs (parameter count vs. accuracy vs. training time) involved in choosing a CNN's hyperparameters.

## 11. References


1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. PyTorch Documentation.
5. CIFAR-10 Dataset Documentation.